Assignment 3 — Cloud Computing and Microservices Resilience

AITU Digital Student Services Platform

Astana IT University | Fault Tolerance | Academic Year 2026–2027

Scenario

Design a resilient cloud-native architecture for a synthetic Astana IT University digital student-services platform. The platform supports timetable lookup, course information, registration requests, and student authentication.

Architecture: Users → Load Balancer → API Gateway → Auth / Course / Registration → Database.

Given requirements

Parameter	Requirement
Students/users	10,000
Peak requests	500 req/s
Availability target	99.9%
Maximum RTO	15 min
Maximum RPO	5 min
Maximum API latency	500 ms
Auth replicas	2
Course replicas	2
Registration replicas	2
Database nodes	2

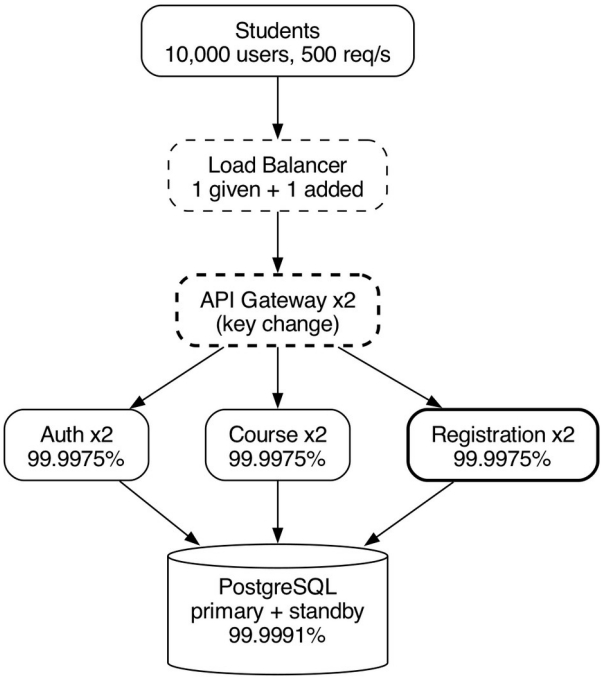
Synthetic component availability

Component	Availability
Load Balancer	99.95%
API Gateway	99.90%
Auth instance	99.50%
Course instance	99.50%
Registration instance	99.50%
Database node	99.70%

Part A — Architecture Analysis (5 points)

Component	Failure scenario	Effect on student service	Resilience mechanism
Load Balancer	Instance crash, misconfiguration, zone outage, traffic flood during the registration period	Single entry point: **the whole platform is unreachable**, even though every backend is healthy (single point of failure, 99.95%)	Two LB instances active-active, or a managed multi-zone cloud LB. Health checks on the gateways. DNS failover with a short TTL. Rate limiting.
API Gateway	Process crash or hang, overload at 500 req/s, bad routing config after a deploy	**All API calls fail** (login, timetable, registration). It is the weakest link in series: 99.90% on its own uses the whole 99.9% budget	**2+ replicas** behind the LB (key change). Liveness and readiness probes. Rolling updates with maxUnavailable: 0. Timeouts and circuit breakers towards the backends so that one

			slow service cannot block the gateway.
Auth	One replica crashes. Signing-key or config error. Auth DB unavailable.	New logins fail. If tokens are validated by calling Auth, every request fails.	**2 replicas** + probes + PDB. **Stateless JWT**: the other services validate tokens locally with the public key, so users who are already logged in keep working while Auth is down.
Course	One replica crashes, slow DB queries	Timetable and course-information lookup fail or are slow	**2 replicas** + probes. Read-heavy data is cacheable (cache or CDN with a TTL; serve stale timetable data rather than an error).
Registration	Replica crash, deadlock or hang, DB connection exhaustion, DB unavailable	Students cannot submit or view registrations. This is the most critical function during the registration week (peak load).	**2 replicas** on different nodes (anti-affinity). Readiness probe tied to the DB. Liveness independent of the DB. PDB minAvailable: 1. Fast-fail 503 + Retry-After, so clients retry instead of hanging. Idempotent requests are recommended for safe retries.
Database	Primary node failure, disk full, data corruption, human error (DROP TABLE)	All writes fail (registration). Reads fail for every service. Corruption or human error means **data loss**.	**2 nodes**: primary + streaming standby in another zone, with automatic failover (Patroni or CloudNativePG). Connection pooling. **Backups with history** (base backup + WAL archive → PITR) for errors that replication would copy.



Architecture diagram (docs/architecture.png). Dashed = redundancy added by this design.

Part B — Availability Calculation (7 points)

For two independent replicas use Rparallel = 1 − (1 − R1)(1 − R2). Calculate each redundant subsystem and approximate end-to-end availability.

Subsystem	Configuration	Availability
Auth	2 replicas	$1 - (1 - 0.995)^2 = 1 - 0.000025 =$ **99.9975%**
Course	2 replicas	$1 - (1 - 0.995)^2 = 1 - 0.000025 =$ **99.9975%**
Registration	2 replicas	$1 - (1 - 0.995)^2 = 1 - 0.000025 =$ **99.9975%**
Database	2 nodes	$1 - (1 - 0.997)^2 = 1 - 0.000009 =$ **99.9991%**
End-to-end	Series	$0.9995 \times 0.9990 \times 0.999975^3 \times 0.999991 =$ **99.8417%**

1. Does the design meet 99.9%?

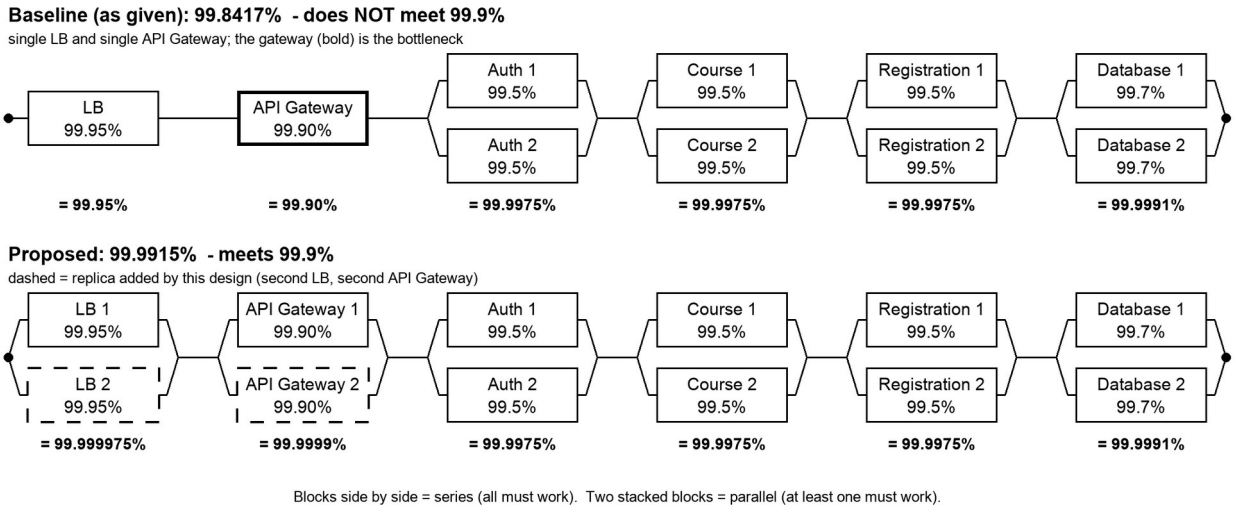
No. 99.8417% means about **13.9 h of downtime per year** (68 min per month). The target 99.9% allows only 8.76 h per year (43 min per month).

2. Which component is the bottleneck?

The **API Gateway** (single instance, 99.90%). It causes 63% of the expected downtime, and the single LB causes another 32%. On its own the gateway already equals the target, so any other component in series pushes the result below 99.9%.

3. What redundancy change would have the largest effect?

A second API Gateway replica. End-to-end availability rises from 99.8417% to **99.9415%** (**5.12** h downtime per year), which meets 99.9%. With 2 Load Balancers as well: 99.9915%. For comparison, 3 replicas of every service give only 99.8491% and 3 database nodes only 99.8426%.



Reliability block diagram: blocks side by side are in series, two stacked blocks are replicas in parallel.

How the availability was calculated

1. Downtime per year: (1 − A) × 8,760 h. Baseline: 0.001583 × 8,760 = 13.87 h (68 min per month). The 99.9% target allows 8.76 h. With 2 API Gateways: 5.12 h. With 2 LBs + 2 API Gateways: 0.75 h.

2. Bottleneck: each subsystem's share of the total unavailability (1 − A). Total ≈ 0.0010 (Gateway) + 0.0005 (LB) + 3 × 0.000025 (services) + 0.000009 (DB) = 0.001584. API Gateway: 0.0010 / 0.001584 = 63%; LB: 32%; each service: 1.6%; Database: 0.6%. Redundancy therefore pays off on the gateway and the LB, not on the services that already have 2 replicas.

Reproducible with python3 scripts/availability.py (output in results/availability.md).

Part C — Kubernetes Resilience Design (7 points)

Create Kubernetes YAML for the synthetic Registration Service. Do not use real AITU credentials, endpoints, or infrastructure.

- 2 replicas
- Readiness probe
- Liveness probe
- Resource requests and limits
- Rolling update
- Automatic restart
- Service discovery

Mechanism	Implemented?	Purpose
Replicas	Yes: replicas: 2, anti-affinity and topology spread across nodes	One replica can fail without an outage. Gives the 99.9975% subsystem availability from Part B.
Liveness probe	Yes: GET /healthz/live, every 10 s, 3 failures	Restarts a hung or deadlocked process. Does not depend on the DB, so there is no restart storm.
Readiness probe	Yes: GET /healthz/ready (checks the DB), every 5 s, 2 failures	Removes a replica that cannot work from the Service endpoints within about 10 s, so users are not

		routed to it.
Resource limits	Yes: requests 100m CPU / 128Mi, limits 500m / 256Mi	Requests guarantee scheduling capacity. Limits protect the node from a leaking or runaway pod (noisy neighbour).
Rolling update	Yes: RollingUpdate, maxUnavailable: 0, maxSurge: 1, minReadySeconds: 5	Zero-downtime deploys: a new pod must be Ready and stable before an old one is removed. A broken release stalls instead of replacing healthy pods; roll back with kubectl rollout undo.
Service discovery	Yes: ClusterIP Service registration-service → DNS registration-service.aitu-sss.svc.cluster.local	The gateway uses a stable name, not pod IPs. Only Ready pods are endpoints, which gives load balancing plus health-aware routing.
Automatic restart	Yes: restartPolicy: Always; the ReplicaSet recreates deleted pods	Crashed containers are restarted by the kubelet. Lost pods (node failure, deletion) are recreated by the controller.
PodDisruptionBudget	Yes: minAvailable: 1	Node drains and cluster upgrades can never evict both replicas at once.
graceful shutdown	Yes: preStop sleep 5 + SIGTERM handler that fails readiness first	Requests in flight are not dropped during deploys or scale-down.

YAML files: k8s/11-registration-deployment.yaml (Deployment), k8s/12-registration-service.yaml (Service), k8s/13-registration-pdb.yaml (PDB). Validated with kubeconform -strict (11 resources, 0 invalid).

```
spec:
  replicas: 2
  minReadySeconds: 5
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }
  template:
    spec:
      restartPolicy: Always
      terminationGracePeriodSeconds: 30
      affinity: { podAntiAffinity: ... topologyKey: kubernetes.io/hostname }
      containers:
        - name: registration
          image: aitu-demo/registration-service:1.0.0
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits:   { cpu: 500m, memory: 256Mi }
          startupProbe:   { httpGet: { path: /healthz/live,  port: http }, periodSeconds: 2, failureThreshold:
12 }
          readinessProbe: { httpGet: { path: /healthz/ready, port: http }, periodSeconds: 5, timeoutSeconds: 2,
failureThreshold: 2 }
          livenessProbe:  { httpGet: { path: /healthz/live,  port: http }, periodSeconds: 10, timeoutSeconds:
2, failureThreshold: 3 }
          lifecycle: { preStop: { exec: { command: ["sleep", "5"] } } }
---
kind: Service        # registration-service.aitu-sss.svc.cluster.local:80 -> :8080
spec: { type: ClusterIP, selector: { app: registration-service }, ports: [{ port: 80, targetPort: http }] }
```

Part D — Failure Simulation (6 points)

Run/simulate: one Registration replica failure, all Registration replicas failing, and database unavailability.

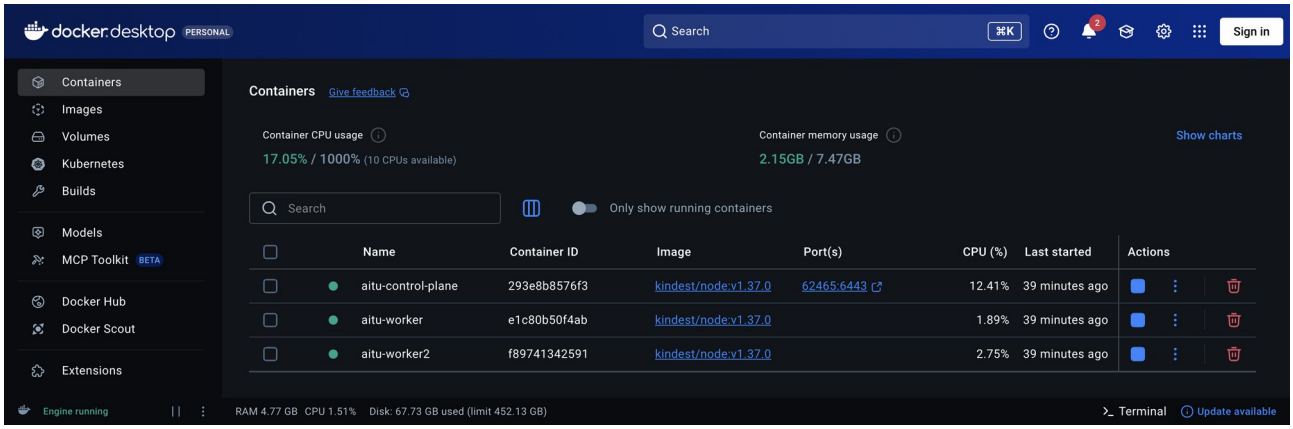
Experiment	Injected failure	Detection time	Recovery time	User impact
1	One Registration	**0.11 s** (pod	**3.02 s** (ReplicaSet	**0 of 35 requests**

	replica	removed from endpoints)	created a new pod, Ready)	**failed.** The second replica served all traffic. Registration stayed available.
2	All Registration replicas	**0.14 s** (0 ready endpoints)	**3.05 s** (both new pods Ready)	**Full outage for 1.25 s**: 7 of 31 requests failed with 000 (connection failed).
3	Database unavailable	**10.03 s** (readiness failed 2 × 5 s, both pods removed from endpoints)	**64.76 s** from injection = **about 5 s after the DB returned**	Registration unavailable for the whole outage (63.9 s): 80 of 101 requests failed. Until detection users got **fast HTTP 503** (20 requests); after both pods left the endpoints, connections failed (000, 60 requests). **0 pod restarts**, because liveness is independent of the DB.

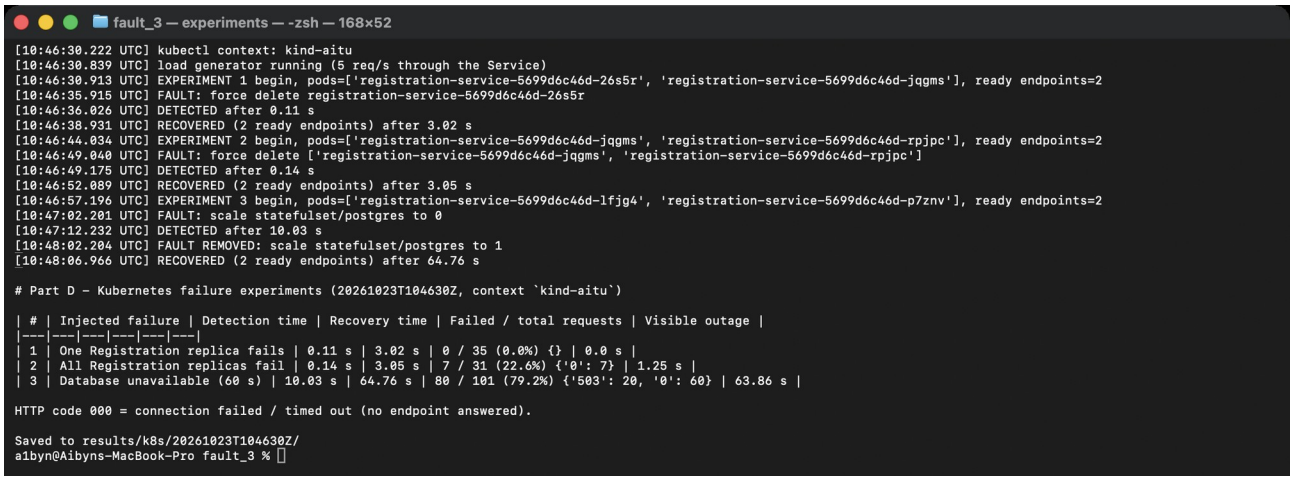
Method. All experiments were run on a real local Kubernetes cluster: kind v0.33, Kubernetes v1.37, 1 control plane + 2 workers (scripts/k8s/setup_kind.sh). The two Registration replicas run on different workers (anti-affinity). The experiment scripts in scripts/k8s/experiments/ (one_replica.py, all_replicas.py, database_down.py; shared code in helpers.py) start an in-cluster load generator (k8s/tools/load-generator.yaml) that sends **5 req/s** of POST /api/v1/registrations through the Service, injects the fault with kubectl, and polls the Service EndpointSlices every ~0.3 s. Detection = fault → ready endpoints drop; recovery = fault → 2 ready endpoints again. User impact is counted from the load-generator log (000 = connection failed, no endpoint answered).

Experiment 4 (bonus): One replica hangs (POST /chaos/hang, liveness stops answering). Detection **≈ 29 s** (3 liveness failures × 10 s → "failed liveness probe, will be restarted"); recovery **≈ 34 s** (container restarted, Ready again). User impact: **None**: the other replica kept serving; the kubelet restarted the hung container automatically (RESTARTS 0 → 1).

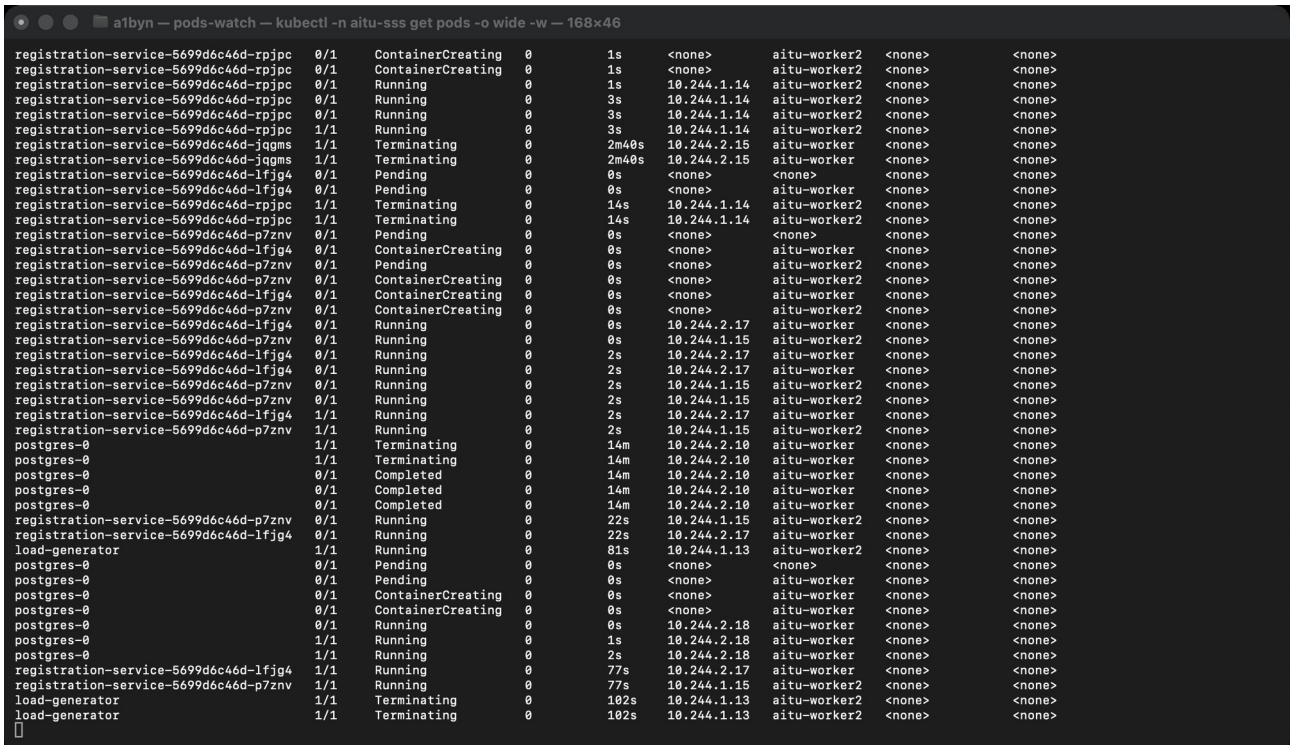
What happens if both replicas fail. The Service has 0 ready endpoints, so the gateway gets connection errors or 503 and registration is down. Recovery is automatic. The kubelet restarts crashed containers (restartPolicy: Always), or the ReplicaSet creates new pods if the pods or the node are gone; the scheduler places them on another node. Traffic returns as soon as the first pod passes readiness (3.05 s for both pods on my kind cluster; longer if the image must be pulled on a new node). Repeated crashes are slowed by CrashLoopBackOff. The PDB protects only against *voluntary* disruptions, not crashes. To reduce the risk: anti-affinity across nodes and zones, a friendly 503 + Retry-After from the gateway, alerting on "0 ready endpoints", and a fix for the root cause (for example a bad release → kubectl rollout undo).



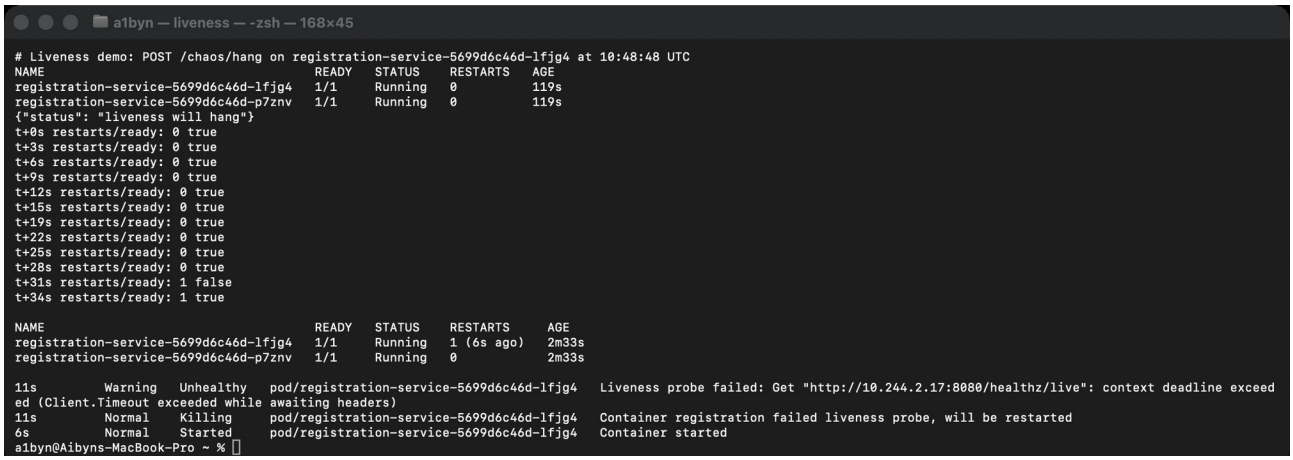
Docker Desktop: the three kind nodes of my cluster running.



Terminal: python3 scripts/k8s/run_experiments.py --db-outage 60 (Experiments 1-3).



Terminal: kubectl -n aitu-sss get pods -o wide -w during the experiments.



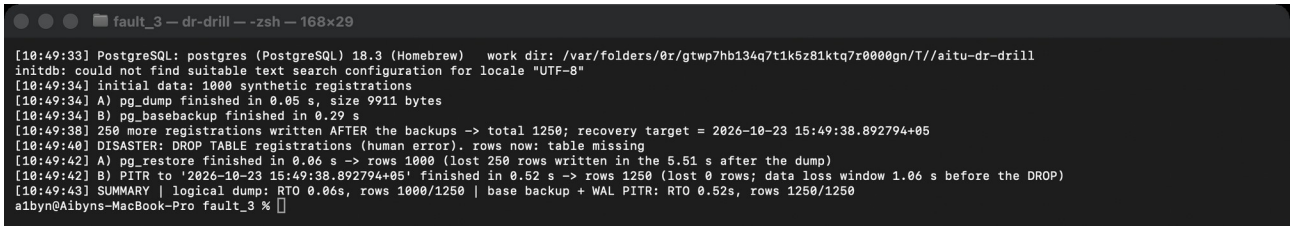
Terminal: Experiment 4, the hung pod fails its liveness probe and is restarted.

Part E — RPO/RTO and Disaster Recovery (5 points)

Mechanism	Frequency	Expected RPO	Recovery time	Purpose
Database backup	Base backup daily at 02:00. WAL continuous (archive_timeout 60 s). Dump every 5 min.	≤ 1 min (WAL), ≤ 5 min (dump)	5–10 min (restore + WAL replay); lab PITR 0.52 s	Data loss, corruption, human error, ransomware, loss of both DB nodes
Replica	Continuous	≈ 0–5 s	30–60 s	HA for DB node or zone failure
Configuration backup	On every change; etcd hourly	0	3–5 min (kubectl apply -f k8s/)	Rebuild the platform, roll back bad configuration
Full recovery	DR drill every semester	≤ 5 min	≤ 15 min (2 + 3 + 2 + 5 + 2 min)	Loss of the cluster or region

Explain why high availability and disaster recovery are different engineering objectives.

HA keeps the service **running** through expected component failures (a pod, node, DB instance or zone). It uses redundancy and automatic failover, is measured in availability % and failover seconds, and needs no human action. DR **brings the service back** after the system or its data has been lost (a region outage, corruption, human error, ransomware). It uses backups with history, PITR, configuration in Git, a standby site and a tested runbook, and it is measured by **RPO and RTO**. HA copies the *current* state to every replica, so it also copies mistakes: in my drill DROP TABLE would have reached the standby within milliseconds. Only DR keeps the *history* needed to go back. Conversely, backups alone give poor availability, because every restore takes minutes. The design needs both.



Terminal: bash scripts/dr_drill.sh: the logical dump restores 1,000 of 1,250 rows, PITR restores 1,250 of 1,250.

Submission

- Architecture diagram: Part A, docs/architecture.png
- Availability calculations: Part B, scripts/availability.py
- Kubernetes YAML: Part C, k8s/11-registration-deployment.yaml, k8s/12-registration-service.yaml
- Failure experiment results: Part D, results/k8s/
- RPO/RTO plan: Part E, docs/dr-plan.md
- Oral defense: notes in docs/defense_notes_ru.md

STUDENT TECHNICAL WORK REPORT / SUBMISSION RECORD

Assignment 3 — Cloud Computing and Microservices Resilience

Purpose: This form is a formal submission record. The student must complete every applicable field and attach evidence of their own work. Short generic explanations are not sufficient.

1. Student and Submission Information

Required field	Student entry
Student full name	Aibyn Talgatov
Student ID	255308
Group / cohort	_____
Instructor	Serek Azamat
Date of submission	23.10.2026
Individual variant / task code assigned by instructor	_____
Git repository / project link	https://github.com/a169n/fault_3, branch main
Attached code / file names	k8s/11-registration-deployment.yaml, k8s/12-registration-service.yaml, k8s/13-registration-pdb.yaml, k8s/20-db-backup-cronjob.yaml, app/registration_service.py, app/config.py, app/store.py, app/handlers.py, scripts/availability.py, scripts/k8s/experiments/*.py, scripts/dr_drill.sh, docs/screenshots/*.png, results/*

2. Work Period and Task Record

Item	Student entry
Work start date/time	12.10.2026, 20:12 (Astana time)
Work completion date/time	23.10.2026
Main task completed	Resilient architecture for the synthetic AITU student-services platform: availability model, Kubernetes resilience design for the Registration Service, failure experiments, RPO/RTO and DR plan
Tools / technologies used	Kubernetes (Deployment, Service, PodDisruptionBudget, CronJob, StatefulSet, probes), Docker, kind, kubectl, kubeconform, Graphviz (diagrams), Python 3, PostgreSQL 16 in the cluster, PostgreSQL 18 for the DR drill (pg_dump, pg_basebackup, WAL archiving, PITR), Git
Main technical output produced	Kubernetes YAML (validated against the Kubernetes 1.31 schema), a Registration microservice with liveness and readiness endpoints, an availability calculator, failure experiments on a real Kubernetes cluster (kind), a DR drill script, measured results

3. Report Structure to Complete

For each section below, write the requested evidence in your own words. Include calculations, diagrams, code references, screenshots, logs, or experimental evidence where applicable.

1. Architecture Analysis

Analyze the failure scenarios for each component and explain the resilience mechanism selected for your design.

Required content	Student response / evidence
Summary	The services behind the gateway are already redundant (2 replicas each). The real single points of failure are the **Load Balancer and the API Gateway**: both are single instances in the given design. The database is redundant against node failure, but replication does not protect against logical errors, so DR backups are needed as well.

Evidence / calculation / artifact reference	docs/architecture.png, docs/rbd.png, table above. For the Registration Service the mechanisms are implemented in k8s/11-registration-deployment.yaml, k8s/12-registration-service.yaml and k8s/13-registration-pdb.yaml.
Student-specific decision or observation	The liveness probe does **not** check the database; only the readiness probe does. If liveness depended on the DB, a DB outage would make the kubelet restart every healthy Registration pod in a loop (a restart storm), which adds load and hides the real cause. Experiment 3 confirms this: the DB was down for 60 s and there were **0 pod restarts**.

2. Availability Calculation

Show the parallel-redundancy calculations and the end-to-end series calculation. Explain whether the target is met and identify the bottleneck.

Required content	Student response / evidence
Summary	The given design reaches 99.84%, which does not meet 99.9%. The bottleneck is the non-redundant API Gateway, followed by the LB. Duplicating the gateway raises availability to 99.94% and meets the target. Duplicating the LB as well gives 99.99%.
Evidence / calculation / artifact reference	scripts/availability.py → results/availability.md; docs/rbd.png
Student-specific decision or observation	Adding replicas to services that are already redundant is almost useless: going from 2 to 3 replicas of every service gains only 0.0074 pp. Redundancy should be spent on the weakest serial link. The model assumes independent failures. In reality replicas share nodes, zones and configuration (correlated failures), so anti-affinity and multi-zone placement are needed for the formula to hold.

3. Kubernetes Resilience Design

Include the Registration Service YAML or a clear excerpt/reference. Explain replicas, probes, resources, rolling update, restart, and service discovery.

Required content	Student response / evidence
Summary	The Registration Service runs 2 replicas with startup, readiness and liveness probes, resource requests and limits, a zero-downtime rolling update, automatic restart, a ClusterIP Service for discovery, a PDB, and graceful shutdown.
Evidence / calculation / artifact reference	k8s/11-registration-deployment.yaml, k8s/12-registration-service.yaml, k8s/13-registration-pdb.yaml. Validation: kubeconform -strict -summary k8s/*.yaml → "Valid: 11, Invalid: 0". Application health endpoints: app/handlers.py (the service is split into config.py settings, store.py PostgreSQL access, handlers.py HTTP endpoints and registration_service.py start-up and graceful shutdown). The DB-backed behaviour was tested against a real PostgreSQL 16 in results/postgres_backend_check.log.
Student-specific decision or observation	**Tuned setting: the readiness probe.** The Kubernetes default (periodSeconds: 10, failureThreshold: 3) needs about 30 s to take a broken replica out of rotation. I set 5 s × 2, which gives about 10 s. **Measured: 10.03 s** in Experiment 3 on the kind cluster. I did not use 1 failure, so that a single slow check (2 s timeout) does not make the endpoint flap.

4. Failure Simulation

Document all three required simulations with detection time, recovery time, and user impact. Explain how each result was obtained.

Required content	Student response / evidence
Summary	With 2 replicas, losing one replica caused no failed

	requests. Losing both caused a 1.25 s full outage, limited by pod re-creation plus readiness time. A DB outage cannot be masked by service replicas: readiness removed the pods within 10 s, and the service recovered on its own about 5 s after the DB returned, with no restarts.
Evidence / calculation / artifact reference	results/k8s/20261023T104630Z/: results.md and results.json (numbers), timeline.txt (fault / detection / recovery times), load-generator.log (every request with timestamp and HTTP code), events.txt (Kubernetes events), pods.txt, liveness_hang.txt (Experiment 4). Screenshots of the run: docs/screenshots/01-05 (shown above).
Student-specific decision or observation	(1) On a real cluster the full outage in Experiment 2 was only 1.25 s because the ReplicaSet immediately created new pods and the startup + readiness probes passed in about 3 s. If the containers were crashing instead (same pod restarted), **CrashLoopBackOff** would add 10 s, 20 s, 40 s … delays, so "two replicas" does not mean "two independent recoveries". (2) During the DB outage users first got a clean 503 from the pods, then connection errors once no pod was Ready. A gateway that returns a friendly 503 + Retry-After when the upstream has no endpoints would make the whole outage look the same to students.

5. RPO/RTO and Disaster Recovery

Complete the RPO/RTO plan and explain why high availability and disaster recovery are different engineering objectives.

Required content	Student response / evidence
Summary	RPO ≤ 5 min is met by continuous WAL archiving (≤ 1 min), with 5-minute logical dumps as a second copy. RTO ≤ 15 min is met only with a warm standby cluster and a scripted runbook. Building a new cluster from nothing would exceed the budget.
Evidence / calculation / artifact reference	scripts/dr_drill.sh → results/dr_drill.log: logical restore 0.06 s, **1,000 of 1,250 rows (250 lost)**; base backup + WAL PITR 0.52 s, **1,250 of 1,250 rows (0 lost)**. k8s/20-db-backup-cronjob.yaml (5-minute dumps, concurrencyPolicy: Forbid, retention of 24 files).
Student-specific decision or observation	The drill showed that "a backup every 5 minutes" is not a guarantee: everything after the last dump is lost (250 rows). PITR from the WAL archive lost nothing. A problem I hit: pg_basebackup took **95 s** the first time because it waited for a spread checkpoint. With -c fast it took 0.29 s in the final run, which matters for the RTO budget.

6. Final Architecture Decision

State the most important resilience change you made and justify it using availability, failure, or recovery evidence.

Required content	Student response / evidence
Summary	**The most important change: run the API Gateway with 2 replicas** (behind 2 LBs or a managed multi-zone LB). It is the bottleneck of the series model; it alone uses the whole 99.9% budget.
Evidence / calculation / artifact reference	End-to-end availability rises from **99.8417% to 99.9415%** (downtime from 13.9 to 5.1 h per year), the largest effect of any single change (Part B, what-if table). Adding replicas to the already-redundant services gives only +0.0074 pp. Experiments 1–2 show the same principle at service level: with 2 replicas, losing one caused 0 failed requests, losing both caused an outage.
Student-specific decision or observation	Redundancy alone is not enough. It must be combined with correct health signals (readiness tied to dependencies, liveness not), anti-affinity against correlated failures, and DR backups with history, because replication copies mistakes.

4. Individual Work and Anti-Plagiarism Record

Because the assignment uses a common educational scenario and dataset, the submitted report must contain verifiable evidence of the individual student's work. Copying another student's wording, calculations, architecture, code, logs, screenshots, or conclusions is not acceptable.

- Use the instructor-assigned individual variant/task code.
- Provide the exact YAML filename or repository commit used for the reported configuration.
- Include evidence from your own simulation or test run.
- Explain one Kubernetes setting you changed or tuned and why.
- Be prepared during oral defense to explain what happens if both replicas fail.

Individual evidence	Student entry
My unique technical decision	Readiness depends on the DB but liveness does not. Readiness tuned to 5 s × 2 (about 10 s detection instead of the default 30 s). maxUnavailable: 0 + minReadySeconds: 5 for zero-downtime rollouts. 2 API Gateways as the key availability change.
My own implementation / calculation evidence	scripts/availability.py output; results/k8s/20261023T104630Z/results.md (my run on a kind cluster on my Mac); results/dr_drill.log.
Exact file, code, commit, notebook, or log reference	k8s/11-registration-deployment.yaml on branch main of the repository above; results/k8s/20261023T104630Z/timeline.txt
One problem I personally encountered and how I solved it	I renamed the database Secret to db-credentials and changed the database name, user and password. After that the stack broke in two places. (1) Three manifests (02-postgres.yaml, 11-registration-deployment.yaml, 20-db-backup-cronjob.yaml) still referenced the old Secret name, so the pods could not get their environment. I updated every secretRef / secretKeyRef. (2) PostgreSQL creates POSTGRES_USER and POSTGRES_DB only when the data directory is empty. The existing volume still had the old user, so the new login would fail. I re-created the lab Postgres volume (synthetic data only) and deleted the old Secret, so nothing could keep using it. Then I verified: both Registration pods Ready, a test registration was stored, current_user = aibyn in the database, and a manual run of the backup CronJob produced a dump. Lesson: changing database credentials needs a migration step (ALTER ROLE or a re-initialised volume), not only a Secret change.
One limitation of my own solution	Part D numbers come from a local kind cluster on one laptop, not from a production cluster: all nodes share one machine, images are preloaded (no pull time), and the load is only 5 req/s. The lab uses a single PostgreSQL pod (the 2-node HA database is described but not deployed), and the pg_dump CronJob writes to a PVC in the same cluster (in production: object storage in another zone).

5. Evidence Checklist

- ☑ Completed required tables and calculations
- ☑ Architecture / RBD / FTA / security diagram where applicable
- ☑ Source code or configuration files where applicable
- ☑ Execution logs / experiment results where applicable
- ☑ Screenshots or other reproducible evidence where applicable
- ☑ Links to code, datasets, notebooks, or artifacts
- ☑ Before/after comparison where applicable
- ☑ Final technical conclusion

6. Student Declaration

I confirm that this report describes my own work. I have not copied another student's report, code, calculations, diagrams, logs, screenshots, or conclusions. Any external source or reused material has been identified. I understand that I may be asked to reproduce or explain any part of this submission during the oral defense.

Role	Name	Signature	Date
Student / Executor	Aibyn Talgatov	a1byn	23.10.2026
Instructor / Accepted by	Serek Azamat	_____	_____

7. Instructor Acceptance Record

Check	Status / comment
Report structure completed	☐ Yes ☐ No Comment: _____
Evidence is sufficient	☐ Yes ☐ No Comment: _____
Individual contribution is identifiable	☐ Yes ☐ No Comment: _____
Oral defense completed	☐ Yes ☐ No Comment: _____
Final status	☐ Accepted ☐ Revision required